ETS

PEMROGRAMAN BASIC ROBOTIKA DENGAN KIT iMCLab



Dosen Pengampu:
Dr. Basuki Rahmat, S.Si. MT

Disusun oleh:
Moh Avin Dharma Wijaya (23081010243)

ROBOTIKA A – 081

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"

JAWA TIMUR

2026

DAFTAR ISI

BAB I PERINTAH DAN KODE PROGRAM 1

1.1 Antarmuka Perintah melalui Komunikasi Serial 2

1.2 Kode Program imclab_naik_turun.ino 2

BAB II HASIL DAN PEMBAHASAN 6

2.1 Prosedur unggah program ke ESP 6

2.2 Pengujian perintah penyalaan motor 6

2.3 Keadaan perangkat pada kondisi motor mati 7

2.4 Keadaan perangkat pada kondisi motor berputar 8

2.5 Pengujian kenaikan kecepatan bertahap 9

2.6 Pengujian penurunan kecepatan bertahap 10

2.7 Pengujian penetapan duty cycle secara langsung 10

2.8 Pengujian pemadaman motor 11

2.9 Ikhtisar seluruh pengujian 11

2.10 Dokumentasi Praktikum 13

BAB III PENUTUP 18

3.1 Kesimpulan 18

3.2 Saran 18

DAFTAR GAMBAR

Gambar 1. Prosedur unggah program ke board ESP32 6
Gambar 4. Diagram rangkaian perangkat pada kondisi motor berputar 9
Gambar 5. Keluaran perintah kenaikan kecepatan pada Serial Monitor 9
Gambar 6. Keluaran perintah penurunan kecepatan pada Serial Monitor 10
Gambar 7. Keluaran perintah penetapan duty cycle langsung pada Serial Monitor 10
Gambar 8. Keluaran perintah pemadaman motor pada Serial Monitor 11
Gambar 9. Kit iMCLab saat program kontrol motor dijalankan dari Serial Monitor 15
Gambar 10. Kit iMCLab dan roda piringan saat motor berputar 17

BAB I PERINTAH DAN KODE PROGRAM

Modul praktikum ini bertujuan membangkitkan sinyal PWM dari board DOIT ESP32 DEVKIT V1 untuk mengatur kecepatan sebuah motor DC pada KIT iMCLab, sekaligus menyalakan sebuah LED sebagai penanda keadaan sistem. Sinyal PWM dibangkitkan pada kanal `pwmChannel` bernilai 0 dengan frekuensi `freq` sebesar 30000 Hz dan `resolution` 8 bit, sehingga lebar pulsa dinyatakan sebagai nilai duty pada rentang 0 sampai 255. Jalur komunikasi yang dipakai adalah serial dengan kecepatan 115200 baud, dan pengguna berinteraksi dengan sistem melalui Serial Monitor. Perangkat keras dihubungkan pada lima titik GPIO, yaitu GPIO 2 untuk `ledPin`, GPIO 27 untuk `motor1Pin1`, GPIO 26 untuk `motor1Pin2`, GPIO 12 untuk `enable1Pin`, dan GPIO 13 yang disediakan bagi `pin_rpm`.

1.1 Antarmuka Perintah melalui Komunikasi Serial

Tujuan perancangan antarmuka ini adalah menyediakan cara yang ringkas bagi pengguna untuk mengubah keadaan motor tanpa menulis ulang program. Sistem menerima lima bentuk perintah yang diketikkan pada Serial Monitor, dan setiap perintah diakhiri dengan karakter baris baru. Tabel berikut merangkum kelima perintah tersebut beserta tugasnya di dalam program.

No	Perintah	Fungsi
1	1	Menyalakan motor. Bila dutyCycle masih bernilai 0, nilai dikembalikan ke nilai default 200
2	0	Mematikan motor dengan menyetel dutyCycle menjadi 0
3	+	Menaikkan dutyCycle sebesar DUTY_STEP (10 satuan), dibatasi DUTY_MAX 255
4	-	Menurunkan dutyCycle sebesar DUTY_STEP (10 satuan), dibatasi DUTY_MIN 0
5	PWM:xxx	Menetapkan dutyCycle secara langsung pada rentang 0 sampai 255

1.2 Kode Program imclab_naik_turun.ino

Kode berikut disalin utuh tanpa perubahan dan merupakan program yang diunggah ke board pada praktikum ini.

```
// ============================================================
// Kontrol Motor DC + LED dengan PWM via Serial
// Perintah:
//   '1'        -> Motor ON (duty terakhir / default)
//   '0'        -> Motor OFF
//   '+'        -> Naikkan kecepatan (+10)
//   '-'        -> Turunkan kecepatan (-10)
//   'PWM:xxx'  -> Set duty cycle langsung (0-255)
// ============================================================
```

```
int ledPin = 2;
char myData = 0;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// Setting PWM properties
const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;
int dutyCycle = 200;         // duty cycle awal
const int DUTY_STEP = 10;    // langkah naik/turun
const int DUTY_MIN  = 0;
const int DUTY_MAX  = 255;

String inputString = "";     // buffer untuk perintah "PWM:xxx"

void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  // konfigurasi PWM
  ledcSetup(pwmChannel, freq, resolution);
  ledcAttachPin(enable1Pin, pwmChannel);

  // pastikan motor mati saat start
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, 0);

  Serial.println("ESP32 Motor Controller siap.");
  Serial.println("Perintah: 1=ON, 0=OFF, +=naik, -=turun, PWM:xxx=set");
}

void applyMotor()
{
  // Terapkan dutyCycle ke driver motor
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);

  // LED indikator: nyala kalau motor bergerak
  digitalWrite(ledPin, dutyCycle > 0 ? HIGH : LOW);
}
```

```
void MotorOn()
{
  // Kalau duty masih 0, pakai default 200
  if (dutyCycle == 0) dutyCycle = 200;
  applyMotor();
  Serial.print("Motor ON, duty = ");
  Serial.println(dutyCycle);
}

void MotorOff()
{
  dutyCycle = 0;
  applyMotor();
  Serial.println("Motor OFF, duty = 0");
}

void naikkanKecepatan()
{
  dutyCycle += DUTY_STEP;
  if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX;
  applyMotor();
  Serial.print("Kecepatan NAIK, duty = ");
  Serial.println(dutyCycle);
}

void turunkanKecepatan()
{
  dutyCycle -= DUTY_STEP;
  if (dutyCycle < DUTY_MIN) dutyCycle = DUTY_MIN;
  applyMotor();
  Serial.print("Kecepatan TURUN, duty = ");
  Serial.println(dutyCycle);
}

void setDuty(int val)
{
  if (val < DUTY_MIN) val = DUTY_MIN;
  if (val > DUTY_MAX) val = DUTY_MAX;
  dutyCycle = val;
  applyMotor();
  Serial.print("Set duty = ");
  Serial.println(dutyCycle);
}

void loop()
{
  while (Serial.available() > 0)
  {
    char c = Serial.read();

    // Cek perintah "PWM:xxx"
    if (c == '\n' || c == '\r')
    {
      inputString.trim();
```

```
      if (inputString.startsWith("PWM:"))
      {
        int val = inputString.substring(4).toInt();
        setDuty(val);
      }
      else if (inputString.length() > 0)
      {
        // perintah satu karakter
        char cmd = inputString.charAt(0);
        if (cmd == '1')      MotorOn();
        else if (cmd == '0') MotorOff();
        else if (cmd == '+') naikkanKecepatan();
        else if (cmd == '-') turunkanKecepatan();
        else {
          Serial.print("Perintah tidak dikenal: ");
          Serial.println(inputString);
        }
      }
      inputString = "";
    }
    else
    {
      inputString += c;
    }
  }
}
```

BAB II HASIL DAN PEMBAHASAN

2.1 Prosedur unggah program ke ESP32

Tahap ini bertujuan memindahkan hasil kompilasi program ke memori flash board agar dapat dijalankan tanpa bergantung pada komputer yang menyusunnya. Perangkat lunak yang dipakai adalah Arduino IDE 2.3.10, dengan pilihan board DOIT ESP32 DEVKIT V1 dan port COM7. Proses unggah dinyatakan berhasil ketika jendela Output menampilkan baris `Writing at 0x…` yang nilainya bertambah dari 20 persen sampai 100 persen disertai keterangan `Uploading…`. Tidak ada pesan kesalahan kompilasi pada rentang tersebut, sehingga seluruh berkas sumber berhasil diterjemahkan menjadi berkas biner. Setelah penulisan selesai, board melakukan reset otomatis dan langsung menjalankan program yang baru diunggah.

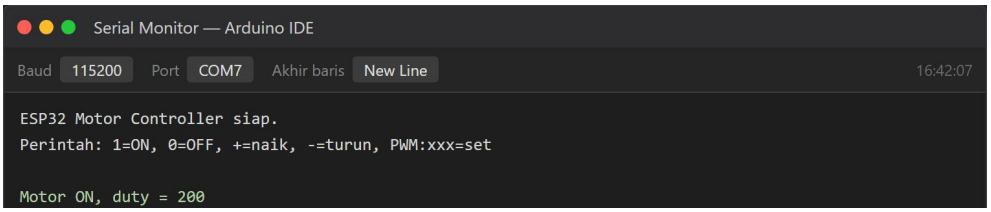


Gambar 1. Prosedur unggah program ke board ESP32

2.2 Pengujian perintah penyalaan motor

Perilaku yang diharapkan dari perintah `1` adalah motor kembali berputar pada nilai duty terakhir yang tersimpan, atau pada nilai default 200 bila sebelumnya motor telah dimatikan. Harapan ini berasal dari isi fungsi `MotorOn()`, yang terlebih dahulu memeriksa kondisi `if (dutyCycle == 0) dutyCycle = 200;` sebelum memanggil `applyMotor()`. Pemeriksaan tersebut membuat perintah `1` tetap bermakna meskipun

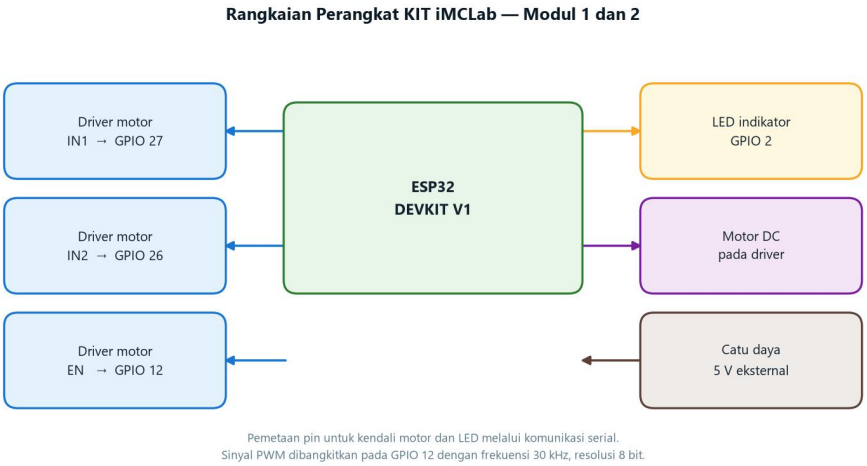
pengguna sebelumnya menekan perintah `0`. Pada pengujian ini Serial Monitor diatur pada 115200 baud dengan akhiran baris New Line, lalu karakter `1` dikirimkan satu kali. Luaran yang tampil adalah `Motor ON, duty = 200`, yang menunjukkan nilai duty berada pada 200 atau setara sekitar 78 persen dari kecepatan maksimum. Respons tersebut sejalan dengan rancangan fungsi.



Gambar 2. Keluaran perintah penyalaan motor pada Serial Monitor

2.3 Keadaan perangkat pada kondisi motor mati

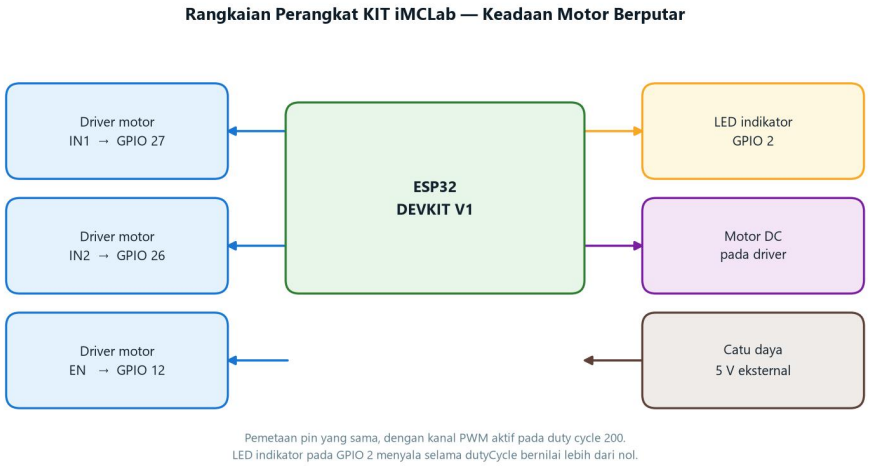
Saat motor tidak diberi daya, dua hal yang perlu ditelusuri adalah kedudukan jalur arah putaran dan keadaan penanda visual sistem. Fungsi `applyMotor()` selalu menuliskan `digitalWrite(motor1Pin1, HIGH)` dan `digitalWrite(motor1Pin2, LOW)`, sehingga jalur arah tidak pernah berubah dan yang berubah hanyalah besar sinyal PWM pada `enable1Pin`. Ketika `dutyCycle` bernilai 0, ekspresi `dutyCycle > 0 ? HIGH : LOW` pada baris `digitalWrite(ledPin, ...)` menghasilkan nilai LOW, sehingga LED indikator pada GPIO 2 padam. Pada kondisi ini tidak ada daya yang disalurkan ke motor dan poros motor tidak berputar.



Gambar 3. Diagram rangkaian perangkat pada kondisi motor mati

2.4 Keadaan perangkat pada kondisi motor berputar

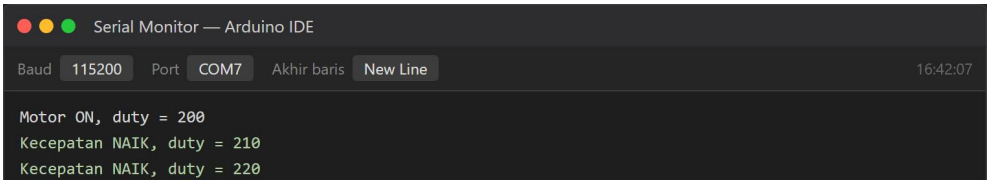
Kebalikan dari kondisi sebelumnya, saat motor mendapat suplai PWM dengan `dutyCycle` di atas nol, penanda visual pada perangkat ikut berubah. Karena `applyMotor()` mengevaluasi `dutyCycle > 0`, LED indikator pada GPIO 2 menyala selama motor bergerak. Dengan cara ini keadaan sistem dapat dikenali langsung dari perangkat keras tanpa harus membuka Serial Monitor. Perlu dicatat bahwa arah putaran motor tetap sama seperti pada kondisi mati, sebab `motor1Pin1` tetap HIGH dan `motor1Pin2` tetap LOW; yang berubah hanyalah kecepatannya.



Gambar 4. Diagram rangkaian perangkat pada kondisi motor berputar

2.5 Pengujian kenaikan kecepatan bertahap

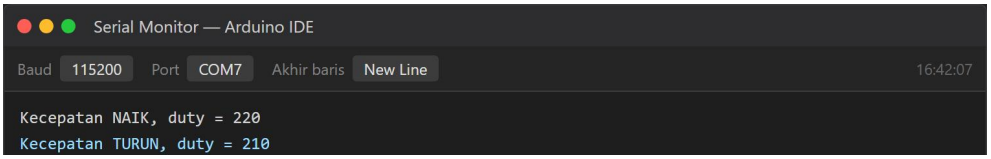
Pengujian ini dimaksudkan untuk memastikan bahwa perubahan kecepatan dapat dilakukan secara bertingkat, bukan hanya secara langsung. Fungsi `naikkanKecepatan()` menambahkan `DUTY_STEP` sebesar 10 pada `dutyCycle` lalu membatasinya dengan `if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX;`. Perintah `+` dikirimkan dua kali setelah motor dinyalakan. Keluaran yang tercatat adalah `Motor ON, duty = 200`, dilanjutkan `Kecepatan NAIK, duty = 210`, dan diakhiri `Kecepatan NAIK, duty = 220`. Selisih antar nilai tepat 10 satuan, sesuai dengan nilai `DUTY_STEP`.



Gambar 5. Keluaran perintah kenaikan kecepatan pada Serial Monitor

2.6 Pengujian penurunan kecepatan bertahap

Pengujian ini merupakan pasangan dari pengujian sebelumnya dan bertujuan memastikan langkah penurunan setara dengan langkah kenaikan. Fungsi `turunkanKecepatan()` mengurangi `dutyCycle` sebesar `DUTY_STEP` dan menahan nilainya dengan `if (dutyCycle < DUTY_MIN) dutyCycle = DUTY_MIN;`. Berawal dari nilai duty 220, satu kali perintah `-` menghasilkan keluaran `Kecepatan TURUN, duty = 210`. Penurunan sebesar 10 satuan ini mencerminkan sifat simetris antara fungsi penambah dan fungsi pengurang kecepatan.



Gambar 6. Keluaran perintah penurunan kecepatan pada Serial Monitor

2.7 Pengujian penetapan duty cycle secara langsung

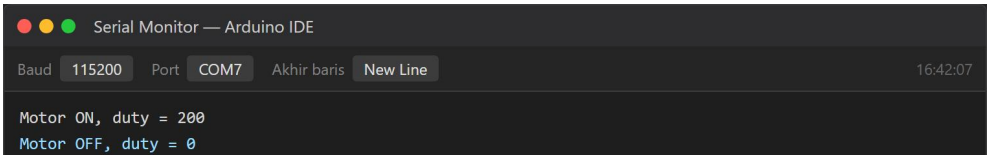
Cara lain untuk menentukan kecepatan adalah dengan menetapkan nilai duty secara langsung, tanpa melalui penambahan atau pengurangan bertahap. Mekanisme ini ditangani fungsi `setDuty(int val)`, yang lebih dahulu menjaga nilai masukan pada rentang `DUTY_MIN` sampai `DUTY_MAX` sebelum menyimpannya ke `dutyCycle`. Perintah `PWM:220` diberikan pada pengujian ini dan dijawab dengan keluaran `Set duty = 220`. Nilai duty berpindah dari 200 ke 220 dalam satu perintah, sehingga cara ini lebih ringkas dibandingkan menekan perintah `+` dua kali. Alur pemrosesannya adalah pemeriksaan `inputString.startsWith("PWM:")` di dalam `loop()`, dilanjutkan pengambilan angka melalui `inputString.substring(4).toInt()` dan pemanggilan `setDuty(val)`.



Gambar 7. Keluaran perintah penetapan duty cycle langsung pada Serial Monitor

2.8 Pengujian pemadaman motor

Pemadaman motor diuji dengan mengirimkan karakter `0`. Fungsi `MotorOff()` menyetel `dutyCycle = 0`, memanggil `applyMotor()`, lalu mencetak pesan bahwa motor telah dimatikan. Keluaran yang tercatat adalah `Motor OFF, duty = 0`, yang muncul setelah keluaran `Motor ON, duty = 200`. Karena `dutyCycle` berada pada nilai 0, LED indikator pada GPIO 2 kembali padam seperti pada kondisi motor mati.



Gambar 8. Keluaran perintah pemadaman motor pada Serial Monitor

2.9 Ikhtisar seluruh pengujian

Tabel berikut memuat rangkuman seluruh pengujian yang telah dibahas.

No	Perintah	Hasil Pengamatan
1	1	Pesan Motor ON, duty = 200, roda berputar, LED menyala
2	+	Pesan Kecepatan NAIK, duty naik dari 200 ke 210 lalu ke 220
3	-	Pesan Kecepatan TURUN, duty turun dari 220 ke 210
4	PWM:220	Pesan Set duty = 220, duty langsung menjadi 220
5	0	Pesan Motor OFF, duty = 0, roda berhenti, LED padam

Selain kesesuaian keluaran serial dengan perilaku perangkat keras, pembacaan menyeluruh terhadap kode program menghasilkan empat catatan yang perlu dibahas.

Pertama, arah putaran motor tidak pernah berubah. Fungsi `applyMotor()` selalu menuliskan `motor1Pin1` sebagai HIGH dan `motor1Pin2` sebagai LOW tanpa syarat, sehingga konfigurasi jembatan H pada driver motor bersifat tetap. Akibatnya program

ini hanya mengatur kecepatan motor, bukan arah putarannya. Untuk dapat membalik arah, salah satu dari kedua pin tersebut harus dapat dipertukarkan nilainya.

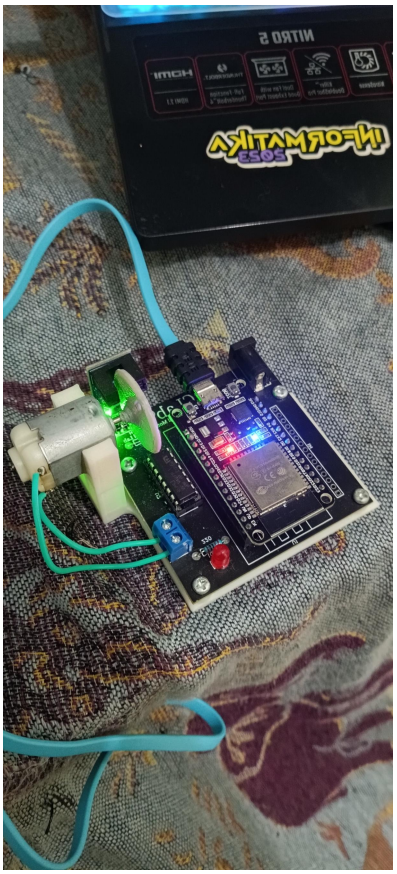
Kedua, buffer `inputString` tidak menerima karakter `\r`. Pada blok percabangan di dalam `loop()`, karakter `\r` bersama `\n` diperlakukan sebagai penanda akhir perintah, sedangkan penambahan karakter ke buffer hanya terjadi pada blok `else` untuk karakter selain keduanya. Pada Serial Monitor di sistem Windows yang mengirimkan pasangan `\r\n`, urutan pemrosesan perintah menjadi bergantung pada karakter mana yang tiba lebih dahulu. Apabila `\r` datang lebih dulu, buffer masih berisi perintah yang lengkap dan perintah tersebut langsung dieksekusi; karakter `\n` yang menyusul kemudian akan memicu evaluasi kedua terhadap buffer yang sudah kosong dan tidak menghasilkan apa pun. Kondisi sebaliknya juga tetap menghasilkan eksekusi tunggal, sehingga perintah tidak dijalankan ganda, tetapi ketergantungan pada urutan kedatangan karakter tetap merupakan perilaku yang perlu dicatat.

Ketiga, parser berbasis objek `String` dengan penambahan karakter satu per satu berpotensi memicu fragmentasi memori pada pemakaian yang berlangsung lama. Setiap operasi `inputString += c` dapat mengakibatkan alokasi ulang pada heap, sehingga dalam jangka panjang ruang memori terpecah menjadi blok-blok kecil. Alternatif yang lebih aman adalah memakai buffer `char` berukuran tetap beserta indeks posisi penulisan, misalnya array dengan panjang terbatas yang diisi hingga karakter akhir perintah ditemukan.

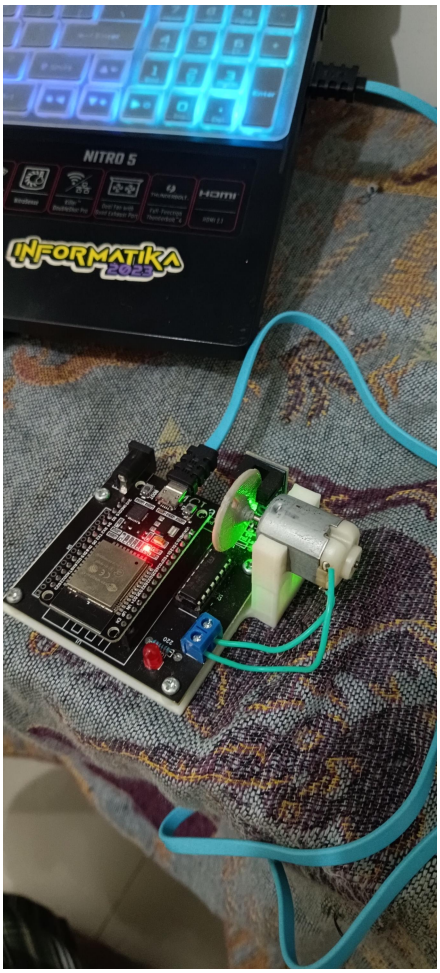
Keempat, fungsi `applyMotor()` memanfaatkan LED indikator pada GPIO 2 sebagai penanda bahwa motor sedang bergerak. Penentuan dinyala-padamkannya didasarkan pada ekspresi `dutyCycle > 0`, sehingga status sistem dapat dibaca secara visual langsung dari perangkat. Pemakaian LED ini tidak mengganggu jalur kendali motor karena berada pada GPIO yang terpisah.

2.10 Dokumentasi Praktikum

Praktikum modul pertama dijalankan menggunakan kit iMCLab dengan board DOIT ESP32 DEVKIT V1. Motor DC pada kit diatur melalui perintah yang dikirim dari Serial Monitor, dan setiap perubahan duty cycle terlihat langsung pada putaran roda serta indikator LED pada GPIO 2. Foto berikut mendokumentasikan keadaan kit ketika perintah kontrol dijalankan.



Gambar 9. Kit iMCLab saat program kontrol motor dijalankan dari Serial Monitor



Gambar 10. Kit iMCLab dan roda piringan saat motor berputar

BAB III PENUTUP

3.1 Kesimpulan

Beberapa hal yang dapat disimpulkan dari praktikum ini adalah sebagai berikut.

3.2 Saran

Untuk pengembangan berikutnya, arah putaran motor dapat ditambahkan sebagai perintah tersendiri dengan cara membuat nilai `motor1Pin1` dan `motor1Pin2` dapat dipertukarkan, sehingga fungsi modul tidak terbatas pada pengaturan kecepatan. Penanganan akhiran baris juga sebaiknya diseragamkan, misalnya dengan hanya memakai `\n` sebagai penanda akhir perintah, agar perilaku program tidak bergantung pada sistem operasi pengirim. Selain itu, penggantian objek `String` dengan buffer `char` berukuran tetap layak dipertimbangkan untuk mengurangi risiko fragmentasi memori pada pemakaian jangka panjang. Sebagai pelengkap, pembacaan sensor RPM pada `pin_rpm` dapat diaktifkan agar kecepatan putaran dapat diverifikasi secara kuantitatif, tidak hanya melalui nilai duty.